

# Design Patterns | HKU

Date: 16/05/25

Name: R de Fost

Project: [Design Patterns for GDV - Game Architecture](#)

Github: [Design-Pattern-Exercise](#)

## Project Info

**Description:** a proof of concept spell casting mechanic, to summon specialized minions to use in a battle.

### Controls:

arrow keys [Cast spell]

Spacebar [Confirm spell]

Spell Input:

|         |         |         |           |            |
|---------|---------|---------|-----------|------------|
| 1: Body | 2: Body | 3: body | 4: Attack | 5: Defense |
|---------|---------|---------|-----------|------------|

Spell Combinations

|         | Water | Earth | Fire  | Air   |
|---------|-------|-------|-------|-------|
| Body    | ↓↓↓   | ↑↑↑   | ←←←   | →→→   |
| Attack  | ↑ (5) | ↓ (4) | ← (4) | → (2) |
| Defense | ↑ (5) | ↓ (4) | ← (4) | → (2) |

## Explanation:

The gameplay is slightly inspired by the magic system from the game [Magicka](#) and how by combining different elements a new type of spell is created.

These spells are created from basic attributes that the elements possess and by combining them it adds secondary typings and effects to the current primary element.

Besides the combinations, there are also a few set combinations that will produce a 'special' effect that has a specific combination of elements to create.



## Motivation:

Command: The idea of the game is to also work with controllers, so this will be a lot easier to implement later down development. Furthermore, the spell mechanic can have multiple effects that need to happen during the casting, so using a command pattern it will be a lot more flexible to implement any needed functionality.

ObjectPool: The intended gameplay is relatively fast, so there will be a lot of enemies spawning and dying during the game, so using an objectpool would help a lot with performance.

Decorator: Because of the combining of inputs, there will be a lot of different types of minions, so by using decorators only the individual components and attributes of the minions will have to be made and can easily be changed or swapped during development.

## UML:

